

UNIVERSIDAD AUTÓNOMA DE CAMPECHE

Seguridad Informática

PROFESOR: Sergio A Noh Puch

(ITS) A

7 SEMESTRE

NOMBRE: ANGEL ANTONIO PEREZ REYES



UAC

**Universidad Autónoma
de Campeche**

"Del enigma sin albas a triángulos de luz"

5 de Septiembre de 2026

1. Objetivo

Generar las claves del algoritmo RSA con los primos proporcionados y usar la clave privada para descifrar el mensaje.

Lo resolví con un programa en Python.

2. Datos de entrada

Los datos de la práctica son $p = 67$ y $q = 83$.

El mensaje cifrado es: 1058, 2597, 4955, 4201, 3162, 4343, 2754, 2497, 336, 2597

3. Desarrollo

3.1 Paso 1: Calcular n

$$n = p \times q = 67 \times 83 = 5561$$

3.2 Paso 2: Calcular $\varphi(n)$

$$\varphi(n) = (p - 1) \times (q - 1) = 66 \times 82 = 5412$$

3.3 Paso 3: Elegir e

Empecé a probar números desde el 2 y en cada uno saqué el máximo común divisor con 5412. Si el resultado no es 1 quiere decir que comparten un divisor y por eso se descarta. La práctica pide el cuarto que sirva, así que fui anotando todos los que probé:

e	mcd(e, 5412)	Resultado
2	2	Se descarta
3	3	Se descarta
4	4	Se descarta
5	1	Candidato 1

6	6	Se descarta
7	1	Candidato 2
8	4	Se descarta
9	3	Se descarta
10	2	Se descarta
11	11	Se descarta
12	12	Se descarta
13	1	Candidato 3
14	2	Se descarta
15	3	Se descarta
16	4	Se descarta
17	1	Candidato 4

Los que sirvieron fueron 5, 7, 13 y 17. Como me piden el cuarto, entonces $e = 17$ y la clave pública queda como (17, 5561).

3.4 Paso 4: Calcular d

Aquí hay que encontrar el número d que al multiplicarlo por 17 y sacarle el módulo 5412 me dé 1. Usé el algoritmo extendido de Euclides. Primero hice las divisiones:

$$5412 = 17(318) + 6 \quad 17 = 6(2) + 5 \quad 6 = 5(1) + 1 \quad 5 = 1(5) + 0$$

El último residuo que no es cero es 1, entonces sí existe el inverso. Después sustituí hacia atrás:

$$1 = 6 - 5(1) \quad 1 = 6 - [17 - 6(2)] = 3(6) - 17 \quad 1 = 3[5412 - 17(318)] - 17 \quad 1 = 3(5412) - 955(17)$$

De ahí me sale que -955×17 me da 1 en módulo 5412, pero como es negativo le sumo 5412:

$$d = -955 + 5412 = 4457$$

Entonces la clave privada es (4457, 5561).

Para comprobar: $17 \times 4457 = 75769$, y 75769 entre 5412 da 14 de cociente y sobra 1, que es lo que se buscaba.

3.5 Paso 5: Descifrar el mensaje

La fórmula para descifrar es $M = C^d \bmod n$, o sea cada número elevado a la 4457 y sacándole módulo 5561. Cada resultado es un número de la tabla ASCII que luego se convierte en letra.

Bloque	C	M	Letra
1	1058	72	H
2	2597	79	O
3	4955	76	L
4	4201	65	A
5	3162	32	espacio

6	4343	77	M
7	2754	85	U
8	2497	78	N
9	336	68	D
10	2597	79	O

Juntando las letras el mensaje es: **HOLA MUNDO**

Nota: los bloques 2 y 10 son el mismo número y por eso dan la misma letra. Eso pasa porque se cifró letra por letra, así que una letra repetida siempre da el mismo número.

4. Resumen de resultados

Dato	Valor
p	67
q	83
n	5561
$\varphi(n)$	5412
e	17
d	4457
Clave pública	(17, 5561)
Clave privada	(4457, 5561)
Mensaje	HOLA MUNDO

5. Evidencias

5.1 Código fuente

```
> Users > angpe_downhvs > Downloads > practica2_rsa.py > ...
1  # Practica 2 - Algoritmo RSA
2  # Alumno: Angel Antonio Perez Reyes
3  # Seguridad Informatica
4  # Imparte: Sergio Noh Puch
5
6  p = 67
7  q = 83
8
9  n = p * q
10 phi = (p - 1) * (q - 1)
11
12 print("PASO 1 y 2: CALCULO DE n Y phi(n)")
13 print("p =", p)
14 print("q =", q)
15 print("n = p * q =", p, "*", q, "=", n)
16 print("phi(n) = (p-1) * (q-1) =", p - 1, "*", q - 1, "=", phi)
17
18
19 def mcd(a, b):
20     while b != 0:
21         residuo = a % b
22         a = b
23         b = residuo
24     return a
25
26
27 print()
28 print("PASO 3: BUSQUEDA DE e")
29 print("Condiciones: 1 < e < phi(n) y mcd(e, phi(n)) = 1")
30 print()
31
32 contador = 0
33 numero = 2
34 e = 0
35
```

```

35
36 while contador < 4:
37     divisor = mcd(numero, phi)
38
39     if divisor == 1:
40         contador = contador + 1
41         e = numero
42         print("e =", numero, " mcd =", divisor, " --> CANDIDATO", contador)
43     else:
44         print("e =", numero, " mcd =", divisor, " --> no sirve")
45
46     numero = numero + 1
47
48 print()
49 print("El cuarto candidato es e =", e)
50
51
52 print()
53 print("PASO 4: BUSQUEDA DE d (algoritmo extendido de Euclides)")
54 print("Se busca d tal que", e, "* d mod", phi, "= 1")
55
56
57 def euclides_extendido(a, b):
58     # devuelve (mcd, x, y) tal que a*x + b*y = mcd
59     if b == 0:
60         return a, 1, 0
61     g, x1, y1 = euclides_extendido(b, a % b)
62     x = y1
63     y = x1 - (a // b) * y1
64     return g, x, y
65
66
67 g, x, y = euclides_extendido(e, phi)
68
69
67 g, x, y = euclides_extendido(e, phi)
68 d = x % phi
69
70 print("mcd(e, phi) =", g)
71 print("Coeficiente x (antes de ajustar) =", x)
72 print("d = x mod phi =", d)
73 print("Comprobacion:", e, "*", d, "=", e * d)
74 print(e * d, "mod", phi, "=", (e * d) % phi)
75
76 print()
77 print("CLAVE PUBLICA = (" , e, ", ", n, ")")
78 print("CLAVE PRIVADA = (" , d, ", ", n, ")")
79
80
81 print()
82 print("PASO 5: DESCIFRADO DEL MENSAJE")
83 print("Formula: M = C^d mod n")
84 print()
85
86 cifrado = [1058, 2597, 4955, 4201, 3162, 4343, 2754, 2497, 336, 2597]
87
88 mensaje = ""
89
90 for c in cifrado:
91     m = pow(c, d, n)
92     letra = chr(m)
93     mensaje = mensaje + letra
94     print("C =", c, " -> M =", m, " -> letra =", letra)
95
96 print()
97 print("MENSAJE DESCIFRADO:", mensaje)
98

```

5.2 Resultados del programa

```
PS C:\Users\angpe_3ownhvs\Downloads> python practica2_rsa.py
PASO 1 y 2: CALCULO DE n Y phi(n)
p = 67
q = 83
n = p * q = 67 * 83 = 5561
phi(n) = (p-1) * (q-1) = 66 * 82 = 5412

PASO 3: BUSQUEDA DE e
Condiciones: 1 < e < phi(n) y mcd(e, phi(n)) = 1

e = 2 mcd = 2 --> no sirve
e = 3 mcd = 3 --> no sirve
e = 4 mcd = 4 --> no sirve
e = 5 mcd = 1 --> CANDIDATO 1
e = 6 mcd = 6 --> no sirve
e = 7 mcd = 1 --> CANDIDATO 2
e = 8 mcd = 4 --> no sirve
e = 9 mcd = 3 --> no sirve
e = 10 mcd = 2 --> no sirve
e = 11 mcd = 11 --> no sirve
e = 12 mcd = 12 --> no sirve
e = 13 mcd = 1 --> CANDIDATO 3
e = 14 mcd = 2 --> no sirve
e = 15 mcd = 3 --> no sirve
e = 16 mcd = 4 --> no sirve
e = 17 mcd = 1 --> CANDIDATO 4

El cuarto candidato es e = 17

PASO 4: BUSQUEDA DE d (algoritmo extendido de Euclides)
Se busca d tal que 17 * d mod 5412 = 1
mcd(e, phi) = 1
Coeficiente x (antes de ajustar) = -955
d = x mod phi = 4457
Comprobacion: 17 * 4457 = 75769
75769 mod 5412 = 1

CLAVE PUBLICA = ( 17 , 5561 )
CLAVE PRIVADA = ( 4457 , 5561 )

PASO 5: DESCIFRADO DEL MENSAJE
Formula: M = C^d mod n
```

```

El cuarto candidato es e = 17

PASO 4: BUSQUEDA DE d (algoritmo extendido de Euclides)
Se busca d tal que 17 * d mod 5412 = 1
mcd(e, phi) = 1
Coeficiente x (antes de ajustar) = -955
d = x mod phi = 4457
Comprobacion: 17 * 4457 = 75769
75769 mod 5412 = 1

CLAVE PUBLICA = ( 17 , 5561 )
CLAVE PRIVADA = ( 4457 , 5561 )

PASO 5: DESCIFRADO DEL MENSAJE
Formula: M = C^d mod n

C = 1058 -> M = 72 -> letra = H
C = 2597 -> M = 79 -> letra = O
C = 4955 -> M = 76 -> letra = L
C = 4201 -> M = 65 -> letra = A
C = 3162 -> M = 32 -> letra = 
C = 4343 -> M = 77 -> letra = M
C = 2754 -> M = 85 -> letra = U
C = 2497 -> M = 78 -> letra = N
C = 336 -> M = 68 -> letra = D
C = 2597 -> M = 79 -> letra = O

MENSAJE DESCIFRADO: HOLA MUNDO
PS C:\Users\angpe_3ownhvs\Downloads>

```

5.3 Comparación

Los valores que obtuve con el programa coinciden con los cálculos manuales del método de Euclides, y la comprobación de que $e \times d$ en módulo $\phi(n)$ da 1 se cumple correctamente.

6. Conclusiones

En conclusión, logré generar las claves del algoritmo RSA: la pública (17, 5561) y la privada (4457, 5561), con las que descifré el mensaje "HOLA MUNDO". El paso que más me costó entender fue calcular d usando el algoritmo extendido de Euclides; hacer las sustituciones hacia atrás es fácil de confundir con los signos, y de hecho me salió un valor negativo (-955) que tuve que ajustar sumándole $\phi(n)$ para obtener el equivalente positivo dentro del módulo. También me quedó claro cómo funciona la seguridad de RSA, si alguien intercepta la clave pública (17, 5561), no puede obtener la clave privada directamente porque necesita conocer $\phi(n)$, y para eso tendría que factorizar n en sus primos originales p y q. Con primos tan pequeños como estos, factorizar 5561 es polémico, pero con primos de cientos de dígitos ese proceso es computacionalmente inviable, y ahí es donde está la seguridad real del algoritmo.

